BATCH 2 — Vehicle to Vehicle Communication using Machine Learning & Traffic Simulation

(You can literally start your viva with the below introduction)

---

1. How to Start Explaining (Opening Statement)

"Our project is Vehicle-to-Vehicle Communication based Intelligent Traffic Management System.
The aim of the system is to reduce traffic congestion, predict best route for drivers, identify accident severity and dynamically control traffic signals using Artificial Intelligence and simulation."

This project is not a single ML model.
It is a **combination of 3 intelligent subsystems**:

1. Route Prediction (ML based)
2. Accident Severity Prediction (ML based)
3. Smart Traffic Signal Control (Simulation using PyGame)

The system simulates how future smart cities will manage traffic automatically without human traffic police.

---

2. Problem Statement

Current traffic systems have major problems:

• Drivers choose wrong route → congestion
• Traffic signals are fixed timer based → unnecessary waiting
• Accidents are reported late → delayed emergency services
• No communication between roads and vehicles

Because of this:

- fuel is wasted
- pollution increases
- ambulances get delayed
- travel time increases

---

3. Objective

The objective of the project is:

1. Suggest best route to driver based on traffic & weather
2. Automatically adjust signal timing based on vehicle density
3. Predict accident severity using historical accident data
4. Provide intelligent transportation decision support

---

4. Technologies Used (and WHY)

Technology	Why Used
Python	Core programming language for ML and simulation
Machine Learning (Scikit-Learn)	To predict routes and accident severity
Random Forest	Best for classification with high accuracy
Decision Tree	Fast prediction and interpretable model
SVM	Comparison algorithm

Technology	Why Used
PyGame	To simulate traffic signal and vehicles visually
Jupyter Notebook	Data analysis and model training
Pandas	Dataset handling
NumPy	Numerical processing
Matplotlib	Graph visualization

---

5. Project Architecture

The system works in **3 layers**:

Layer 1 — Data Layer

Datasets used:
• Weather dataset
• Traffic congestion dataset
• US Accident Severity dataset

SCREENS

Layer 2 — Machine Learning Layer

Models trained:
• Route Prediction Model
• Accident Severity Model

Layer 3 — Simulation Layer

• PyGame Traffic Simulator
• Dynamic signal controller

---

6. Folder & File Explanation

From your screenshot:

Important Files

File	Purpose
Main.py	Main program controller
traffic_simulation.py	Smart signal simulation
test.py	Testing ML predictions
TrafficRoutePrediction.ipynb	Route prediction ML training
runSimulation.bat	Runs traffic simulator

File	Purpose
Dataset/	Training datasets
images/	Vehicle and signal graphics

---

7. Internal Working (VERY IMPORTANT — Viva Question)

Now I will explain what actually happens inside the system.

Step 1 — Route Prediction (Machine Learning)

Input:

- Weather condition
- Traffic congestion
- Direction

Process:

1. Dataset loaded using **Pandas**
2. Date converted → day, month, year
3. String labels (NB, EB, SW etc.) converted to numbers

4. Data normalized
5. Dataset split:
    - 80% Training
    - 20% Testing

Algorithms Used:

- SVM
- Decision Tree
- Random Forest

Why Random Forest works best?

Because:
It combines multiple decision trees → reduces overfitting → improves accuracy.

Your results:

- SVM ≈ Low accuracy
- Decision Tree ≈ High accuracy
- Random Forest ≈ Highest accuracy

Output:

System predicts best route:
Example:

WB = Westbound
NB = Northbound
SE = South-East

So driver gets **alternate less congested route**.

Step 2 — Accident Severity Prediction

Second model is trained using **US Accident dataset**.

Input:

- Weather
- Road condition
- Visibility
- Traffic

Output:

Severity Level:

Value	Meaning
2	Minor accident
3	Moderate accident
4	Severe accident

Internal ML Logic

Random Forest checks:

- feature importance
- probability distribution
- class voting

Every tree predicts severity → majority vote becomes final prediction.

This helps:
→ emergency vehicles (ambulance) get priority route.

Step 3 — Smart Traffic Signal (PyGame Simulation)

This is the most interesting part of your project.

The file:
traffic_simulation.py

What happens inside?

The simulation continuously counts number of vehicles in each lane.

Each lane has:
• Vehicle queue
• Density counter

Algorithm:

1. Vehicles generated randomly
2. Each lane vehicle count calculated
3. Lane with highest count gets green signal
4. Others stay red

Signal Timing:

Instead of fixed 30 seconds,
the signal becomes:

Dynamic Timing

More vehicles → longer green
Less vehicles → shorter green

This is exactly how modern AI traffic systems work.

The simulation runs in **infinite loop** and stops when window closed.

Execution Flow (When you run project)

1. Train ML model in Jupyter Notebook

2. Save trained model
3. Run runSimulation.bat
4. PyGame window opens
5. Vehicles appear
6. Signal automatically adjusts
7. Route prediction tested using test.py
8. Accident severity predicted

Why This Project is Important (Real-World Use)

This project can be used in:

• Smart cities
• Google Maps route suggestion
• Ambulance priority systems
• Highway monitoring systems
• Intelligent Transportation Systems (ITS)

Advantages

• Reduces traffic congestion
• Reduces fuel consumption
• Faster emergency response

• Automatic signal control
• Data-driven decision making

Limitations

• Requires real-time sensors (camera / IoT) in real life
• Simulation only, not hardware connected
• Lane widening not possible via software

Conclusion (How to End)

“In conclusion, our system demonstrates how Artificial Intelligence and Vehicle-to-Vehicle communication concepts can be used to build an intelligent traffic management system that predicts routes, detects accident severity and dynamically controls traffic signals to reduce congestion and improve road safety.”

Possible Viva Questions

Q: Why Random Forest instead of SVM?
Because traffic data is nonlinear and Random Forest

handles complex decision boundaries and gives higher accuracy.

Q: Why PyGame?
To visually simulate real-world traffic behavior and dynamic signal control.

Q: Why normalization?
To bring all features to same scale so ML model learns correctly.

Q: What is confusion matrix?
It compares predicted class vs actual class to evaluate classification performance.